JAVASCRIPT

Objetos

JSON

- Crear objetos
- Acceder a las propiedades
- Agregar propiedades
- Métodos de un objeto
- Crear nuestro toString
- JSON
- JSON.parse(texto)
- JSON.stringify()
- Desestructuración: arreglos, objetos, spread, rest.

Profesora:
Elizabeth Gisela Oviedo Luna

OBJETOS

En Javascript, existe el tipo de dato OBJECT.

Un objeto encapsula información relacionada, para posteriormente poder acceder a ella de forma sencilla e intuitiva.

Crear objetos

El objeto se puede crear con "new" pero además existe una forma más sencilla y directa de hacerlo englobando los campos entre llaves {}.

En JS

```
let jugador = {
  nombre: "Martín",
  puntos:100
};
```

Acceder a las propiedades

Los campos de los objetos suelen llamarse propiedades y podemos acceder a ellas a través de la notación con:

- Puntos

```
console.log("Jugador: "+jugador.nombre+" con "+jugador.puntos+" puntos");
```

- corchetes.

```
console.log("Jugador: "+jugador["nombre"]+" con "+jugador["puntos"]+" puntos");
```

Mostrará en cualquiera de los dos casos:

```
Jugador: Martín con 100 puntos
```

A partir de ahora nos quedaremos con la notación de puntos para manejar objetos pero se podría realizar de igual forma con corchetes.

Agregar propiedades

Para añadir propiedades al objeto después de haberlo creado.

```
let jugador = {};
jugador.nombre= "Luisa";
jugador.puntos=120;
console.log("Jugador: "+jugador.nombre+" con "+jugador.puntos+" puntos");
```

```
Jugador: Luisa con 120 puntos
```

Métodos de un objeto

Podemos agregar una función o una variable que contiene una función, creando un método de un objeto

```
let jugador={
  nombre:"Martín",
  puntos:100,
  saludar:function() {return "Hola! Soy "+this.nombre}
}
```

```
console.log(jugador.saludar());
```

```
Hola soy Martín
```

Crear nuestro toString

El método toString() intenta representar la información de ese objeto en un string, pero no funciona como nosotros

```
console.log(jugador.toString())
```

```
[object Object]
```

Podríamos crear una variable toString dentro de nuestro objeto, que contenga una función que muestre el texto ideal para representar nuestro objeto de información.

```
jugador.toString= function () {
        return `${this.nombre} - Puntos: ${this.puntos}`
    };

console.log(jugador.toString());
```

En consola:

```
Martín - Puntos: 100
```

JSON

JSON siglas de **JavaScript Object Notation**.

Es un formato de texto utilizado para definir estructuras de objetos y listas complejas.

Es utilizado para el intercambio de datos entre aplicaciones debido a la simplicidad de su implementación y la facilidad de lectura por parte de las aplicaciones y las personas.

Es un subconjunto de la notación de objetos literal de JavaScript, por lo cual existen múltiples similitudes entre ambos.

Los archivos JSON tienen extensión jsn y el mínimo archivo posible en JSON es el archivo con {} solamente.

```
> archivoJSON.jsn
1  {
2
3  }
```

Un archivo JSON puede contener cualquier tipo de dato, pero lo más común es que sea un arreglo o un objeto.

Se puede comprobar en https://jsonlint.com/si algo concreto es un JSON válido o no.

Reglas generales:

- Todo el objeto está rodeado por llaves {}.
- Los nombres de las propiedades están rodeados por comillas dobles “”.

- Los tipos de valores posibles son: texto, número, booleano, array, objetos o null.
- Los textos string deben estar entrecomillados con «comillas dobles»
- Las listas o arrays se rodean con corchetes [] y pueden contener cualquiera de los tipos de datos aceptados.
- No es posible guardar tipos como: function, date o expresión regulares.
- No se puede agregar comentarios. Solo formatos derivados de JSON sí permiten comentarios, como es el caso de JSON5.

Ejemplo:

```
{
  "nombre": "Jaime",
  "vidas": 2,
  "puntos": 100,
  "activo": true,
  "poderes": ["rapidez", "fuerza", "invisibilidad"],
  "sentidos": {
    "visión": 50,
    "audición": 75,
  }
}
```

Método JSON.parse()

Convierte un JSON a objeto, esta acción se denomina parsear. Analiza un texto que contiene un JSON válido y devuelve un objeto JavaScript con dicha información correctamente estructurada.

```
let jugadorJSON = `{
  "nombre": "Pablo",
  "vidas": 4
}`;
let jugador = JSON.parse(jugadorJSON);
console.log(jugador.nombre); //Pablo
console.log(jugador.vidas);  //4
```

Método JSON.stringify()

Convertir objeto a JSON.

```
let jugador = {
  nombre: "Pablo",
  vidas: 99,
  saludar: function () {
    return "Hola!";
  },
};

console.log(JSON.stringify(jugador));
```

```
{"nombre":"Pablo","vidas":99}
```

Como las funciones no están soportadas por JSON, por lo que se omite.

Se puede pasar un segundo parámetro que actúa como filtro de campos.

```
console.log("Con ['vidas']->"+ JSON.stringify(jugador,["vidas"]));
console.log("Con ['nombre', 'puntos']->"+JSON.stringify(jugador,["nombre","puntos"]));
console.log("Con []->"+JSON.stringify(jugador,[]));
console.log("Con null->"+JSON.stringify(jugador,null));
```

```
Con ['vidas']->{"vidas":1}
Con ['nombre', 'puntos']->{"nombre":"Rodrigo","puntos":150}
Con []->{}
Con null->{"nombre":"Rodrigo","vidas":1,"puntos":150}
```

Desestructuración

Significa separar una estructura, que en JavaScript es un arreglo o un objeto.

Desestructuración de arreglo

Separar elementos de un array y sacarlo a variables individuales.

```
let numeros, primero, segundo;

numeros = [10, 20, 30];
[primero, segundo] = numeros;
console.log(primero); //10
console.log(segundo); //20

[primero, ,segundo] = numeros;
console.log(primero); //10
console.log(segundo); //30

[,primero,,segundo] = numeros;
console.log(primero); //20
console.log(segundo); //undefined
```

En la parte derecha utilizamos los [] para indicar que se trata de un array, pero en la parte izquierda los utilizamos para indicar que hacemos una desestructuración.

Ejemplo: Intercambio de variables

```
let a = 7;
let b = 5;
console.log("a="+a+", b="+b);
[a, b] = [b, a];
console.log("a="+a+", b="+b);
```

```
a=7, b=5
a=5, b=7
```

Los arreglos sólo se están utilizando para hacer la operación de una sola vez, lo único que termina existiendo son las variables a y b.

Desagrupar (Spread) - Agrupar(Rest)

```
let normal = (param) => console.log(param);
let desagrupar = (param) => console.log(...param);
let agrupar = (...param) => console.log(param);

let arreglo = [1, 2, 3, 4, 5];
console.log("Normal:");
normal(arreglo);
console.log("Desagrupar:");
desagrupar(arreglo);
console.log("Agrupar:");
agrupar(arreglo);
```

```
Normal:
▼ (5) [1, 2, 3, 4, 5] i
    0: 1
    1: 2
    2: 3
    3: 4
    4: 5
    length: 5
  ▶ [[Prototype]]: Array(0)
Desagrupar:
1 2 3 4 5
Agrupar:
▼ [Array(5)] i
  ▶ 0: (5) [1, 2, 3, 4, 5]
    length: 1
  ▶ [[Prototype]]: Array(0)
```

Otro ejemplo: agrupación de varios elementos individuales y desestructuración en un arreglo:

```
let numeros = [10, 20, 30, 40];
let [primero, ...resto] = numeros;
console.log(primero); //10
console.log(resto); //[20, 30, 40];
```

```
10
▼ (3) [20, 30, 40] i
    0: 20
    1: 30
    2: 40
    length: 3
  ▶ [[Prototype]]: Array(0)
```

Ejemplo: Reestructuración de arreglo

```
let subarreglo = [3, 4];
// Usando desestructuración + spread
let conspread = [1, 2, ...subarreglo, 5];  // [1, 2, 3, 4, 5]
console.log(conspread);
// Sin usar desestructuración
let sinspread = [1, 2, subarreglo, 5];     // [1, 2, [3, 4], 5]
console.log(sinspread);
```

```
▶ (5) [1, 2, 3, 4, 5]
▼ (4) [1, 2, Array(2), 5] i
    0: 1
    1: 2
  ▶ 2: (2) [3, 4]
    3: 5
    length: 4
  ▶ [[Prototype]]: Array(0)
```

Desestructuración de objetos

Utilizando la desestructuración de objetos podemos separar en variables las propiedades que teníamos en el objeto:

```
let jugador = {
  nombre: "Sara",
  vidas: 2,
  puntos: 150
}
let {nombre,vidas, puntos } = jugador;
console.log("Nombre: "+nombre);
console.log("Vidas: "+vidas);
console.log("Puntos: "+puntos);
```

```
Nombre: Sara
Vidas: 2
Puntos: 150
```

Podemos volver a estructurar en un objeto, uniendo las diferentes variables en un objeto a la hora de mostrarlo por consola.

```
console.log({nombre,vidas,puntos});
```

```
▼ {nombre: 'Sara', vidas: 2, puntos: 150}
    nombre: "Sara"
    puntos: 150
    vidas: 2
  ▶ [[Prototype]]: Object
```

Si queremos poner otro nombre diferente a los definidos, muestra *undefined*:

```
let {nombre,vidas, puntaje} = jugador;
console.log("Puntaje: "+puntaje);
console.log({nombre,vidas,puntaje});
```

```
Puntaje: undefined
▶ {nombre: 'Sara', vidas: 2, puntaje: undefined}
```

Es posible renombrar las propiedades si lo deseamos:

```
let {nombre,vidas, puntos:puntaje} = jugador;
console.log("Puntaje: "+puntaje);
console.log({nombre,vidas,puntaje});
```

```
Puntaje: 150
▶ {nombre: 'Sara', vidas: 2, puntaje: 150}
```

Otra opción es establecerle un valor por defecto:

```
let {nombre,nivel="inicial"} = jugador;
console.log("Nivel: "+nivel);
console.log({nombre,nivel});
```

```
Nivel: inicial
▶ {nombre: 'Sara', nivel: 'inicial'}
```

Reestructurando nuevos objetos

```
let jugTotal={
  ...jugador,
  nivel:"intermedio"
}
console.log(jugTotal);
```

```
▶ {nombre: 'Sara', vidas: 2, puntos: 150, nivel: 'intermedio'}
```